

Обзор KAN

1. Основные моменты

1.1. Теоретическая основа

Сети Колмогорова–Арнольда (KAN) – альтернатива MLP, основанная на **теореме Колмогорова–Арнольда**: любая непрерывная функция f от n переменных на компакте может быть представлена в виде композиции одномерных функций и операций сложения:

$$f(x) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right),$$

где $\phi_{q,p}$ и Φ_q – непрерывные функции одной переменной.

Недостаток исходной теоремы — внутренние функции $\phi_{q,p}$ могут быть **негладкими** и **необучаемыми**. Авторы KAN обобщили представление на **произвольные глубины и ширины** (не только $2n+1$), что эффективно для гладких композиционных функций, типичных для науки.

1.2. Архитектура и параметризация

KAN имеют два основных отличия от MLP:

- функции активации находятся на ребрах, а не в узлах,
- функции активации являются обучаемыми, а не фиксированными.

Это даёт интерпретируемость и помогает обходить проклятие размерности в задачах со структурой.

KAN заменяют скалярные веса w в MLP на обучаемые одномерные функции $\phi(x)$, размещённые на рёбрах графа вычислений. Узлы KAN выполняют только суммирование входящих сигналов.

Слой KAN с n_{in} входами и n_{out} выходами:

$$x_{l+1,j} = \sum_{i=1}^{n_{in}} \phi_{l,j,i}(x_{l,i}), \quad j=1,\dots,n_{out},$$

где $\phi_{l,j,i}$ — функция на ребре от i -го нейрона слоя l к j -му нейрону слоя $l+1$.

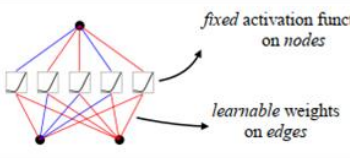
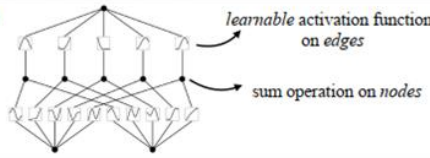
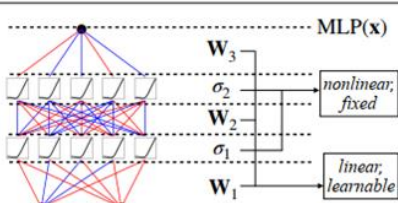
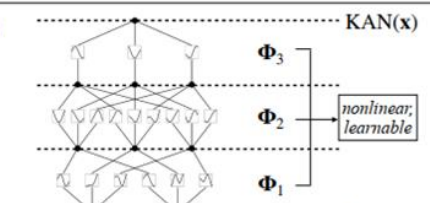
Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(x) \approx \sum_{i=1}^{N(\epsilon)} a_i \sigma(w_i \cdot x + b_i)$	$f(x) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)	(a)  fixed activation functions on nodes learnable weights on edges	(b)  learnable activation functions on edges sum operation on nodes
Formula (Deep)	$MLP(x) = (W_3 \circ \sigma_2 \circ W_2 \circ \sigma_1 \circ W_1)(x)$	$KAN(x) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(x)$
Model (Deep)	(c)  MLP(x) W_3 σ_2 W_2 σ_1 W_1 x nonlinear, fixed linear, learnable	(d)  KAN(x) Φ_3 Φ_2 Φ_1 x nonlinear, learnable

Рисунок 1.1. Многослойные персептроны (MLP) против сетей Колмогорова-Арнольда (KAN)

Параметризация $\phi(x)$ — ключевой реализационный момент. Каждая функция представляется как:

$$\phi(x) = \omega_b b(x) + \omega_s \text{spline}(x),$$

$$\text{spline}(x) = \sum_i c_i B_i(x)$$

где: $b(x)$ — базисная функция (обычно $\text{silu}(x) = x \cdot \sigma(x)$); $B_i(x)$ — В-сплайновые базисные функции порядка k (кубические, $k=3$); c_i — обучаемые коэффициенты сплайна; ω_b, ω_s — обучаемые масштабирующие коэффициенты.

Коэффициенты ω_b и ω_s избыточны (могут быть поглощены c_i и базисом), но оставлены для лучшего контроля величины активации.

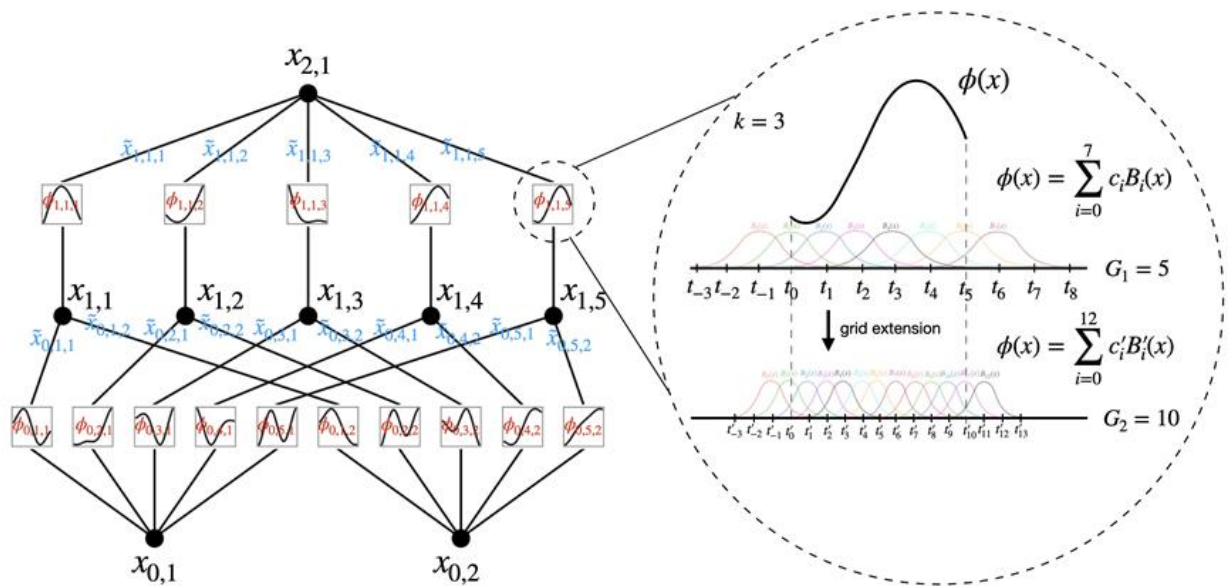


Рисунок 1.2. Слева: Обозначения активаций, проходящих через сеть. Справа: функция активации параметризована как В-сплайн, что позволяет переключаться между крупнозернистыми и мелкозернистыми сетками.

1.3. Инициализация KAN: в руках выполняется в два этапа

1. Коэффициенты В-сплайнов c_i инициализируются из $N(0, \sigma^2)$ с малым σ (обычно $\sigma=0.1$), чтобы $\phi(x) \approx \text{base}(x)$ на старте.
2. Масштабирующие коэффициенты ω_b и ω_s инициализируются по схеме Ксавье (Xavier), как в линейных слоях MLP.

Итог: сеть стартует с разумного поведения, близкого к тождественному преобразованию (Identity-преобразование, выход=вход).

1.4. Обучение: градиенты и оптимизация

KAN обучаются стандартным обратным распространением с оптимизаторами (Adam, LBFGS). Градиенты вычисляются относительно:

- коэффициентов В-сплайнов c_i ;
- масштабирующих коэффициентов ω_b, ω_s .

Особенности:

- При использовании LBFGS с поиском по строке обучение может значительно замедляться при больших G .

- Авторы KAN отмечают, что при $G > 100$ оптимизация может становиться проблемной из-за плохого ландшафта потерь.

Внешние и внутренние степени свободы в KAN различают:

- Внешние — структура графа (как соединены узлы);
- Внутренние — точки сетки сплайнов.

Это различие позволяет независимо управлять архитектурой и точностью аппроксимации.

Общее число параметров для KAN глубины L , ширины N , порядка сплайна k и числа интервалов G примерно: $O(N^2 L(G+k)) \sim O(N^2 L G)$. Для MLP с теми же L и N : $O(N^2 L)$. KAN требуют больше параметров на слой, но на практике требуют значительно меньшей ширины N , что компенсирует разницу.

2. Обновление сетки (Grid Extension)

В-сплайны в KAN определены на сетке с G интервалами. В процессе обучения значения активаций (выходные данные нейрона) могут выходить за пределы этой области. Вне сетки сплайн начинает экстраполировать, что приводит к скачкам градиентов, нестабильности обучения и ухудшению обобщения. Решения:

2.1. Динамическое обновление сетки (на лету)

Сетка каждого сплайна периодически перестраивается под текущее распределение входных активаций. Например, каждые N шагов обучения или при достижении порога по проценту активаций вне сетки система:

- собирает статистику по x -значениям, которые поступают на вход сплайна;
- заново расставляет узлы сетки так, чтобы покрыть актуальный диапазон значений;
- пересчитывает коэффициенты сплайна.

Это предотвращает экстраполяцию и разрушение обучения.

Авторы пробовали альтернативы (нормализацию, градиентное обновление сетки), но текущий подход показал лучшую производительность.

2.2. Расширение сетки (Grid Extension) — увеличение точности

Это процедура перехода от грубой сетки с G интервалами к мелкой с $G' > G$ интервалами. Повышение точности аппроксимации без переобучения с нуля:

1. **Имеем** обученную функцию на грубой сетке: $\phi_G(x) = \sum_{i=1}^{G+k} c_i B_i^G(x)$, где B_i^G — В-сплайновые базисы порядка k (обычно $k=3$), а $G+k$ — число базисных функций (для G интервалов и порядка k).
2. **Строим** новую мелкую сетку с G' интервалами. Число базисных функций становится $G'+k$. Новая функция имеет вид: $\phi_{G'}(x) = \sum_{j=1}^{G'+k} c'_j B_j^{G'}(x)$
3. **Инициализируем** коэффициенты c'_j новой сетки так, чтобы $\phi_{G'}(x)$ как можно точнее приближала $\phi_G(x)$. Для этого решаем задачу **наименьших квадратов**: $\min_{\{c'_j\}} \mathbb{E}_{x \sim p(x)} [\phi_G(x) - \phi_{G'}(x)]^2$

Здесь $p(x)$ — распределение входных активаций (оценивается по данным или текущему батчу). Задача решается аналитически для каждого сплайна независимо.

Это даёт лестничные кривые потерь: при каждом расширении сетки loss резко падает. В отличие от MLP, где увеличение точности требует переобучения с нуля (дорого), KAN позволяют дешево повышать точность без потери уже изученного.

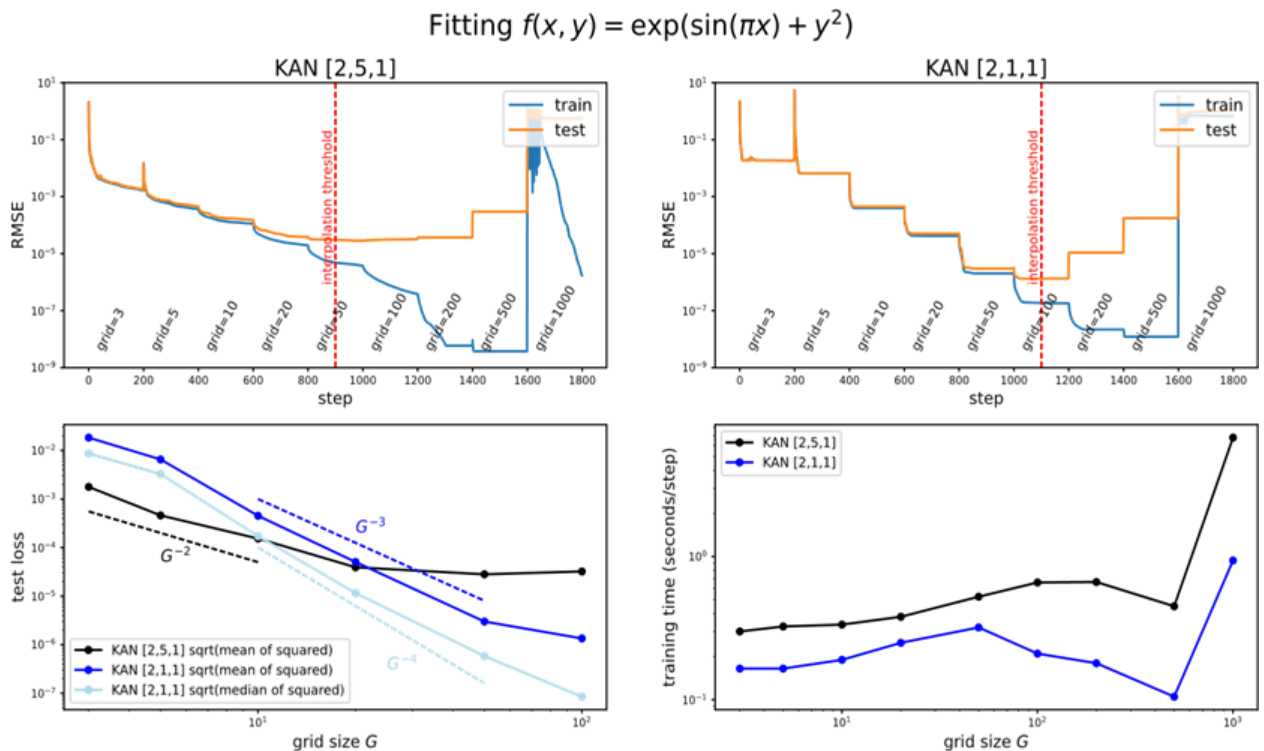


Рисунок 2.1. Мы можем сделать KAN более точными за счет расширения сетки (мелкозернистые сплайновые сетки). Вверху: динамика обучения KAN [2,5,1] ([2,1,1]). Обе модели демонстрируют лестницы на своих кривых потерь, т. е. потери внезапно падают, а затем выходят на плато после расширения сетки. Внизу слева: тест RMSE следует законам масштабирования в зависимости от размера сетки G . Внизу справа: время обучения благоприятно масштабируется с размером сетки G .

2.3. Теория – законы масштабирования сети

Расширение сетки опирается на свойство B-сплайнов: при измельчении сетки пространство сплайнов с G' интервалами **вложено** (или почти вложено) в пространство с G интервалами, если $G' > G$. Это означает, что любая функция, представимая на грубой сетке, может быть хорошо приближена на мелкой — проекция существует и стабильна.

Теоретическая оценка ошибки аппроксимации:

$$\|f - \text{KAN}_G\| \leq C \cdot G^{-k}$$

где k — порядок сплайна. Для кубических сплайнов ($k=3$) это даёт $\alpha=4$, что означает очень быстрое убывание ошибки с ростом G .

На практике KAN[2,1,1] масштабируется как G^{-3} , что близко к теоретическому пределу. Отклонения связаны с:

- **граничными эффектами** — на краях области сплайны ведут себя хуже, чем внутри;
- **неоптимальностью оптимизации** — градиентный спуск не всегда находит глобально оптимальные коэффициенты для мелкой сетки.

Тем не менее, даже с учётом этих отклонений, законы масштабирования KAN значительно быстрее, чем у MLP, особенно в многомерных задачах.

3. Практические инструменты из KAN 2.0 (pykan)

3.1. MultKAN: узлы умножения

В стандартном KAN выходные подузлы слоя становятся входными узлами следующего путём копирования (операция сложения). В KAN 2.0 добавлены узлы умножения, позволяющие явно выявлять мультипликативные структуры.

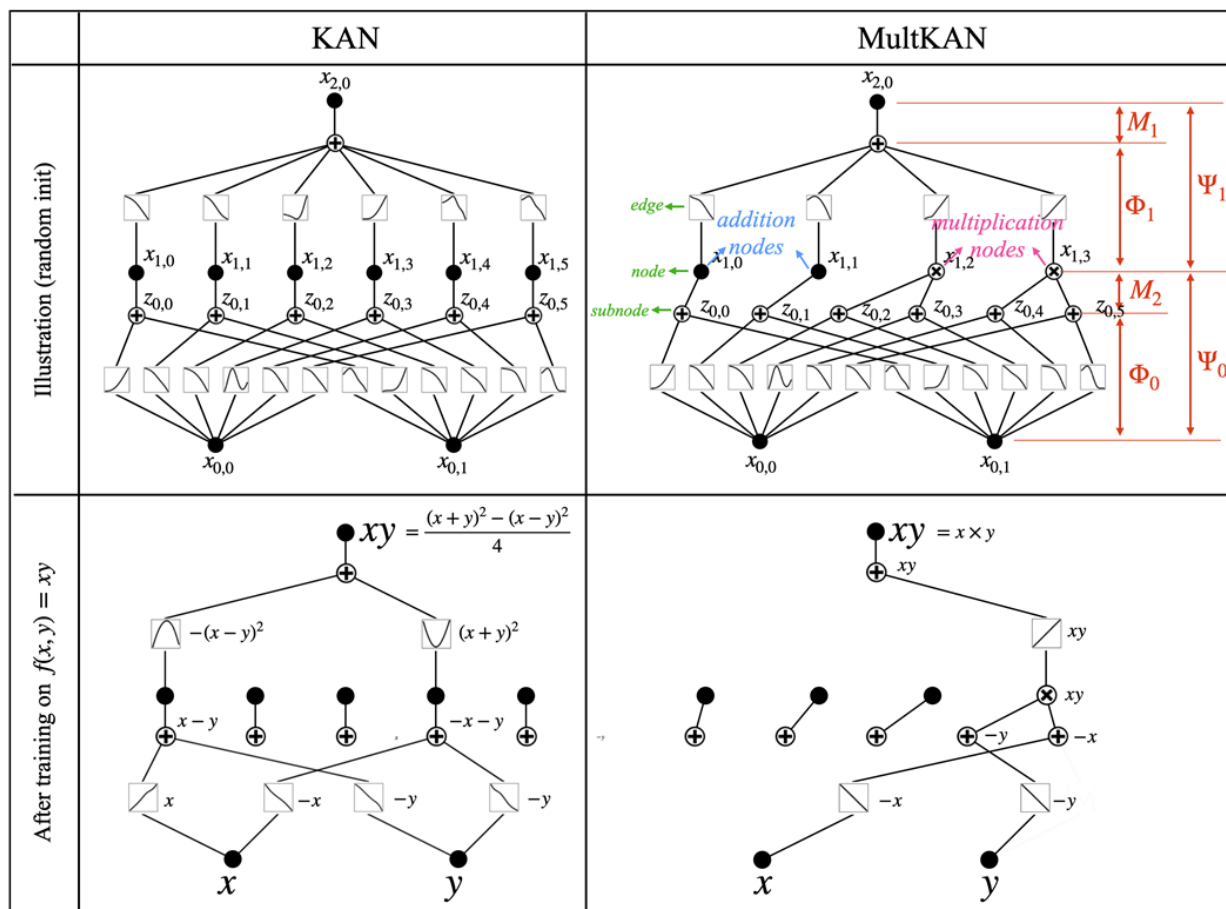


Рисунок 3.1. Вверху: сравнение диаграмм KAN и MultKAN. MultKAN имеет дополнительные слои умножения M. Внизу: После обучения на $f(x, y) = xy$ KAN реализует алгоритм, требующий двух узлов сложения, тогда как MultKAN требует только один узел умножения.

В узлах умножения нет обучаемых параметров — они выполняют только попарное перемножение подузлов предыдущего слоя. Все обучаемые параметры (коэффициенты сплайнов и масштабы) остаются в стандартных KAN-слоях сложения. Благодаря этому все методы разреженной регуляризации из оригинального KAN применимы к MultKAN без изменений.

При нулевом количестве узлов умножения MultKAN вырождается в обычный KAN.

3.2. Встраивание знаний (Science → KAN)

Знания, которые пользователь может встроить в KAN, имеют разные уровни — от самых грубых (какие переменные важны) до самых тонких (точная символическая зависимость):

- **Важные характеристики** — например, утверждение « y полностью определяется x_1 и x_2 , остальные факторы не важны», т.е. существует функция $y=f(x_1, x_2)$.
- **Модульные структуры** — например, « x_1 и x_2 независимо друг от друга аддитивно влияют на y », т.е. $y=g(x_1)+h(x_2)$.
- **Символические формулы** — например, « y зависит от x_1 как синус, от x_2 как экспонента», т.е. $y=\sin(x_1)+\exp(x_2)$.

Для каждого уровня в KAN 2.0 предусмотрены инструменты:

- **Вспомогательные признаки (augment_input)** – пользователь вручную добавляет производные признаки (например, для релятивистской массы вводят $\beta=v/c$ и $\gamma=1/\sqrt{1-\beta^2}$), сокращая число рёбер с 5 до 2–3.
- **Модульные структуры (module)** – пользователь задаёт группы переменных, взаимодействующих только внутри группы (сепарабельность, симметрия). Метод сохраняет внутрикластерные связи, удаляя межкластерные.
- **Компилятор kanpiler** – превращает символьную формулу в KAN с точно заданными функциями за три шага: парсинг формулы в дерево выражений, модификация дерева под структуру графа KAN, объединение переменных в первом слое.

3.3. Извлечение знаний (KAN → Science)

3.3.1. Атрибуция важности признаков (вместо L1-нормы) – оценка вклада каждого ребра в конечный выход через цепное правило. Важность измеряется тем, насколько изменение признака меняет итоговый ответ. Даёт более чистые графики, чем L1, особенно когда активные рёбра в ранних слоях не влияют на выход из-за неактивных последующих.

Инструмент (prune_input) позволяет удалять нерелевантные входы: обучается KAN с D признаками, вычисляются атрибуции, сохраняются только топ- K релевантных, остальные удаляются.

3.3.2. Выявление модульности:

- **Анатомическая модульность (auto_swap)** – перестановка нейронов внутри слоя так, чтобы близкие по функциональности оказывались рядом (на основе паттернов активаций). Сеть становится визуально организованной, видны модули.
- **Функциональная модульность (tree converter)** – использует производные (градиенты, гессианы) для анализа структуры:
 - Определение **разделимости**: аддитивная ($y=g(x1)+h(x2)$), мультипликативная ($y=g(x1)\cdot h(x2)$), симметрии.
 - Построение **дерева**: корень — выход y , внутренние узлы — операции, листья — входы x_i .

Как результат – интерпретируемое дерево решений, позволяющее открывать сохраняющиеся величины, лагранжианы, симметрии и законы.

3.3.3. Улучшенный символьный регресс (три трюка):

А: обнаружить модульность и переобучить с ней:

1. Обучение с регуляризацией → разреженная сеть.
2. Выявление модульности (auto_swap, prune_input).
3. Переобучение с нуля на упрощённой архитектуре (только важные рёбра).

В результате более чистая и точная формула.

В: разреженная инициализация (sparse_init=True):

- Инициализация большинства рёбер нулями, активны лишь некоторые.
- Нулевые рёбра **не замораживаются** — могут обучаться при необходимости.
- Сеть стартует с правильной структуры, наращивая сложность только где нужно.

С: проверка гипотез с системой версий (rewind)

Встроенная система контрольных точек (версии a.b):

1. Из версии 0.1 создаются "параллельные вселенные" для разных гипотез.
2. В каждой зануляются рёбра под конкретную гипотезу → дообучение.
3. `rewind('0.1')` возврат к точке ветвления → версия 0.1 → 1.1, новый раунд.

Улучшения эффективности в KAN 2.0: оптимизирована память с $O(L \cdot N^2 \cdot G)$ до $O(L \cdot N \cdot G)$, добавлена поддержка GPU, отключение символьной ветки и сохранение активаций только при необходимости – ускорение в сотни раз по сравнению с исходной реализацией.

4. Недостатки KAN

- **Медленнее MLP** – примерно в 10 раз (а в некоторых реализациях до 100 раз) из-за вычислений B-сплайнов на каждом ребре, которые сложно распараллелить.
- **Больше параметров при той же ширине** – KAN требуют $O(L \cdot W^2 \cdot (G+k))$ параметров против $O(L \cdot W^2)$ у MLP. Однако на практике KAN часто используют меньшую ширину, что частично компенсирует разницу.
- **Интерпретируемость падает с ростом размера** – для больших сетей визуализация всех функций становится затруднительной, требуются методы агрегации (например, атрибуция или кластеризация рёбер).
- **Требуют структурности данных** – преимущества KAN в полной мере проявляются только когда целевая функция имеет гладкое композиционное представление (как в теореме Колмогорова–Арнольда). На «неструктурных» данных (изображения, текст) KAN уступают MLP.
- **Чувствительность к шуму** – символьная регрессия и сплайновая аппроксимация могут давать нестабильные результаты при наличии даже небольшого шума; стандартная регуляризация не всегда эффективно подавляет осцилляции.
- **Ограниченная область эффективного применения** – KAN уверенно превосходят MLP только в задачах символьной регрессии и научных вычислениях; в классическом ML, компьютерном зрении и NLP они показывают более слабые результаты.
- **Сложность внедрения** – требуют настройки гиперпараметров сплайнов (порядок, число интервалов), динамического обновления сетки, а также плохо масштабируются на крупные сенсорные сети и аппаратные ускорители.

Итог: KAN остаются нишевым, но мощным инструментом для задач, где важна интерпретируемость и структурность данных; для промышленных приложений общего назначения они пока уступают MLP.

5. Итог

KAN — это архитектура, предлагающая иной подход к параметризации нейронных сетей. Их главные преимущества — интерпретируемость (на малых задачах) и способность к итеративному повышению точности через расширение сетки — делают их мощным инструментом для AI+Science и символьной регрессии.

KAN не являются универсальной заменой MLP. В классических задачах машинного обучения, компьютерного зрения и обработки естественного языка MLP остаются более эффективными. KAN — это нишевый, но мощный инструмент, а не революция, отменяющая MLP.

Библиотека `rukan` (KAN 2.0) значительно улучшила эффективность и добавила инструменты для двунаправленной синергии науки и ИИ, что делает KAN ещё более привлекательными для научных исследований. Однако для промышленных приложений общего назначения MLP пока остаются более практичным выбором.